

```
❸ oStudent1.grade = 85
   oStudent2.grade = 92

❹ print(oStudent1.grade)
   print(oStudent2.grade)
```

Listing 8-7: The main code that creates `Student` objects and accesses a property

In the test code, we first create two `Student` objects ❶ and print the grade of each ❷. It looks like we're reaching into each object directly to get the grade values, but since `grade` is a property Python turns these lines into calls to the getter method and returns the value of the private instance variable `self.__grade` for each object.

We then set new grade values for each `Student` object ❸. Here it looks like we're setting values directly into each object's data, but again, because `grade` is a property, Python turns these lines into calls to the setter method. That method validates each value before doing the assignment. The test code ends by printing the new values of the grades ❹.

When we run the test code, we get this output, as we expect:

```
0
0

85
92
```

Using the `@property` and `@<property_name>.setter` decorators gives you the best of both the direct access and getter-and-setter worlds. Client software can be written in a way that *appears* to access instance variables directly, but as the class programmer, your decorated methods get and set the actual instance variables owned by the object and even allow for validation of inputs. This approach supports encapsulation because the client code is not accessing an instance variable directly.

While this technique is used by many professional Python developers, I personally find it a little ambiguous, because when I read other developers' code that uses this approach, it is not immediately apparent whether it's